

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Paulo Henrique Domingues Antunes

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	3
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	4
4.4	Atividade 4 — Controle de acesso por papel — RBAC	5
4.5	Atividade 5 — Logs e auditoria	6
4.6	Atividade 6 — Upload e perfil de usuário	7
5	Atividades extras (opcional)	8
5.1	Atividade E1 — Documentação Swagger/OpenAPI	8
6	Considerações finais	9
7	Declaração de autoria	9

1 Introdução

A disciplina de Introdução à Computação em Nuvem aborda os conceitos fundamentais para a criação, implantação e manutenção de sistemas distribuídos modernos. Ao longo das aulas, o foco principal foi a transição de aplicações locais tradicionais para arquiteturas escaláveis baseadas em microsserviços, utilizando containers e serviços na nuvem. O objetivo foi fornecer uma visão prática e realista dos desafios de infraestrutura, containerização e segurança enfrentados na indústria de tecnologia.

O projeto prático desenvolvido neste bimestre foi o “Catálogo de Filmes — Tom Hanks”, uma aplicação web interativa focada na filmografia do ator. Inicialmente concebido como um catálogo monolítico simples, o sistema evoluiu gradativamente a cada semana. Foram adicionados microsserviços dedicados para autenticação de usuários, controle de acesso baseado em papéis (RBAC), auditoria por logs utilizando Redis Streams, e armazenamento de objetos (Object Storage) com MinIO para fotos de perfil. Este relatório detalha cada etapa dessa evolução, documentando as entregas, prazos, metodologias e evidências técnicas do que foi construído durante o período.

2 Metodologia

O desenvolvimento das atividades seguiu uma abordagem incremental e prática. O ambiente de desenvolvimento baseou-se inteiramente em containers utilizando o Docker e o Docker Compose, permitindo que a infraestrutura (banco de dados MariaDB, servidor Redis, armazenamento MinIO e a própria aplicação) fosse orquestrada localmente com total consistência. O código-fonte foi desenvolvido utilizando Node.js com TypeScript, focando na separação de responsabilidades (Frontend com React, e múltiplos serviços no Backend comunicando-se via rede interna do Docker). As ferramentas de apoio incluíram o Mailtrap para testes de SMTP e o Swagger UI para a documentação da API.

Todas as entregas foram versionadas e publicadas em repositórios públicos no GitHub. As datas e horas apresentadas nas evidências deste relatório foram extraídas de forma rastreável e auditável através do histórico de commits dos próprios repositórios. Utilizando comandos nativos do git (`git log`), foi possível assegurar a honestidade estrutural do relatório, vinculando cada funcionalidade documentada a um ponto exato e imutável na linha do tempo do projeto.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	—	não entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	21/08/2026 15:19	entregue com atraso
3	Desacoplando o login — microsserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	27/08/2026 14:32	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	04/09/2026 16:12	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	25/09/2026 14:41	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	02/10/2026 12:47	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	24/09/2026 18:03	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	—
Situação	não entregue
Evidência	—
Link	—

O que foi feito. Esta atividade não foi entregue.

Dificuldades e como foram resolvidas. A justificativa para a não entrega é que estive ausente na aula presencial onde a atividade foi explicada e construída junto com a turma. Tentei realizar posteriormente em casa, e acredito que não ficou no nível que era o esperado para a atividade, e fiz algo simples.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	21/08/2026 15:19
Situação	entregue com atraso
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. Implementação da base do catálogo interativo consumindo a API externa do TMDb, com layout moderno (Glassmorphism) e integração a um banco de dados MariaDB para favoritar filmes e registrar comentários de forma isolada por usuário.

```
• ~/dev/Projeto-Tom_Hanks on  main 07 x > git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 51865ee
51865ee 21/08/2026 15:19
```

Figura 1: Atividade 2 — evidência da entrega pelo terminal (link, data e hora visíveis)

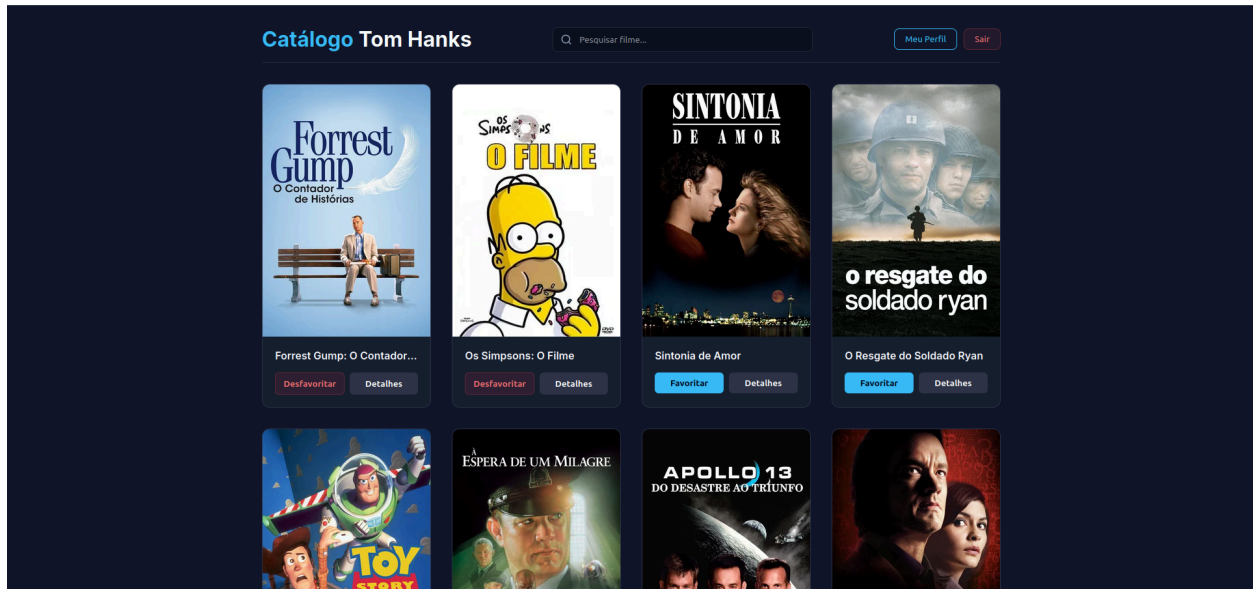


Figura 2: Atividade 2 — resultado (o catálogo funcionando)

Dificuldades e como foram resolvidas. A principal dificuldade foi configurar adequadamente o isolamento de dados no MariaDB e estruturar as variáveis de ambiente para não expor a chave secreta do TMDb. A resolução se deu encapsulando a lógica no backend, que exige estritamente o ID do usuário nas queries.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 14:32
Situação	entregue
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. O sistema foi refatorado e dividido. A lógica de autenticação (cadastro, login com JWT, e recuperação de senha segura) foi separada para um microsserviço independente (auth-service) que roda isolado apenas na rede interna do Docker.

```
• ~/dev/Projeto-Tom_Hanks on  main 04 x > git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 0292a6e
0292a6e 27/08/2026 14:32
```

Figura 3: Atividade 3 — evidência da entrega pelo terminal (link, data e hora visíveis)

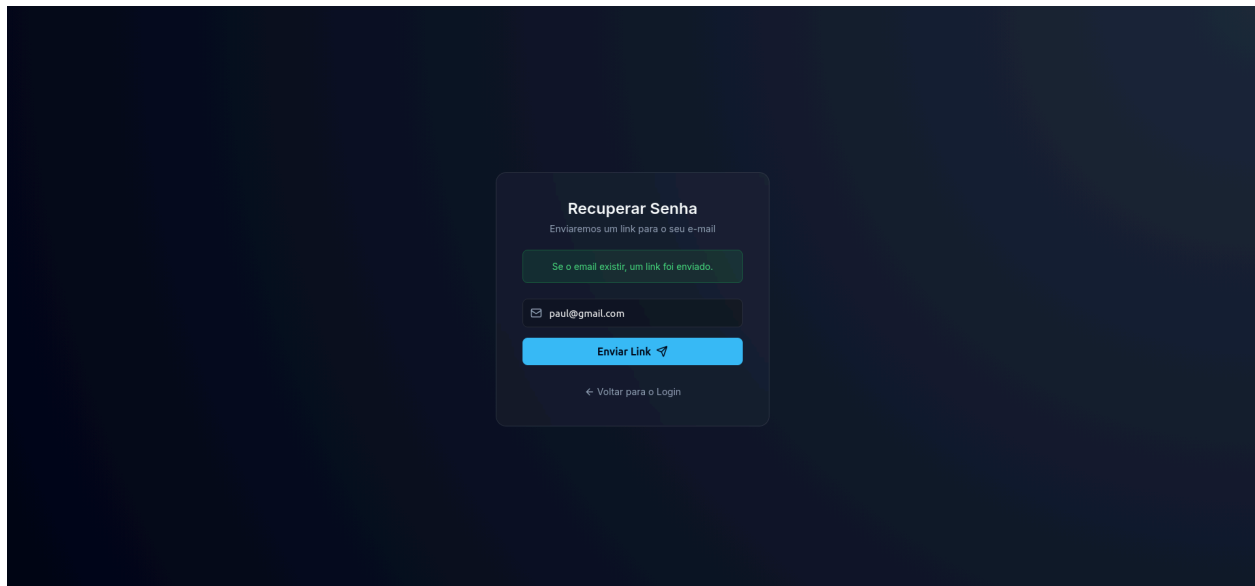


Figura 4: Atividade 3 — resultado (o sistema funcionando)

Dificuldades e como foram resolvidas. Garantir a comunicação entre o catálogo (Gateway) e o serviço interno de autenticação. Resolvido reestruturando o Docker Compose para que apenas o catálogo exponha portas públicas para o navegador, enquanto o roteamento interno repassa as requisições de auth.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	04/09/2026 16:12
Situação	entregue
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. Implementação de controle de autorização injetando a ‘role’ dentro das claims do token JWT (Padrão B). O papel de ‘admin’ ganhou a exclusividade de poder moderar e apagar comentários de qualquer pessoa, gerando erros 403 para acessos não autorizados.

```
• ~/dev/Projeto-Tom_Hanks on main ✖ > git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" a6a1b70
a6a1b70 04/09/2026 16:12
```

Figura 5: Atividade 4 — evidência da entrega pelo terminal (link, data e hora visíveis)

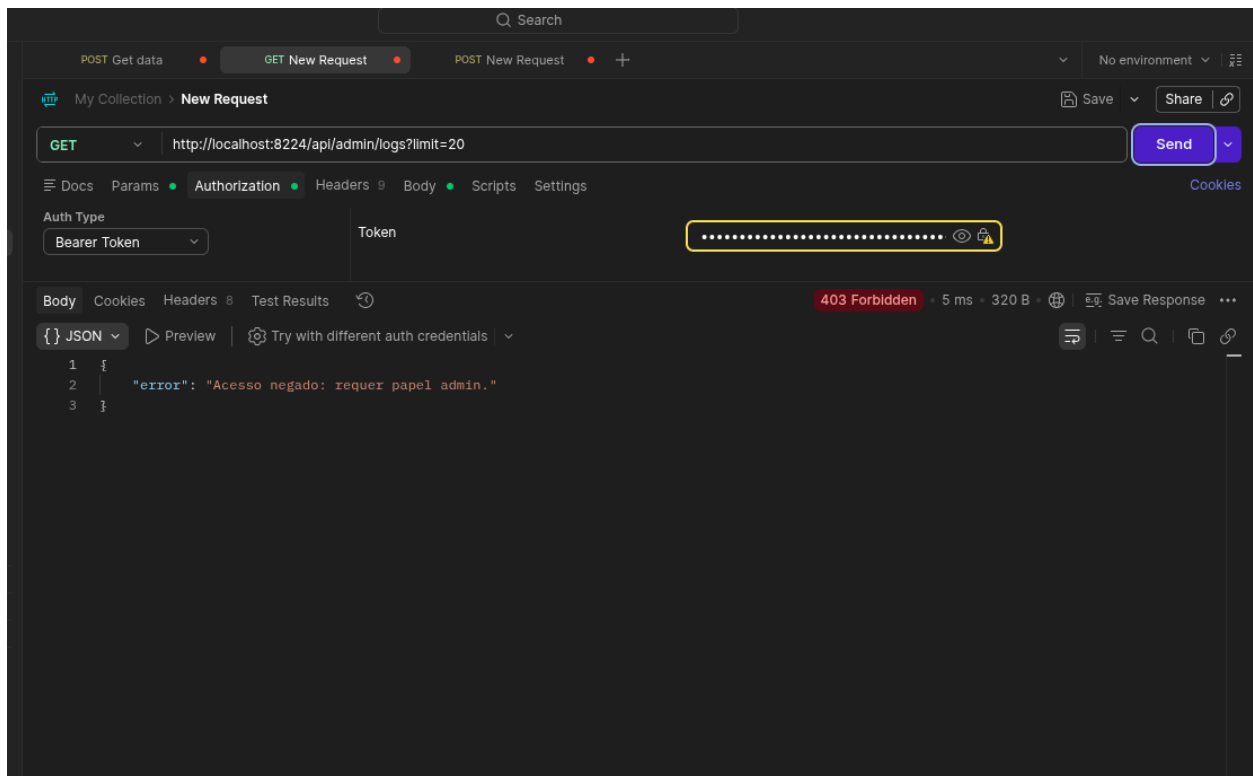


Figura 6: Atividade 4 — resultado (sistema funcionando com 403 visível)

Dificuldades e como foram resolvidas. Implementar a restrição sem sobrecarregar o auth-service com requisições redundantes a cada clique no catálogo. Optou-se pelo Padrão B, onde o Middleware do catálogo valida localmente a assinatura do JWT extraindo os privilégios.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	25/09/2026 14:41
Situação	entregue
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. Criação do log-service, focado em observabilidade. Utilizando o Redis Streams, foi criada uma trilha de auditoria “fire-and-forget” que registra eventos críticos como logins e tentativas de acessos negados.

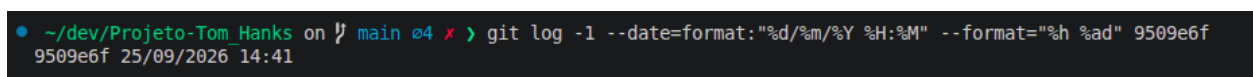


Figura 7: Atividade 5 — evidência da entrega pelo terminal (link, data e hora visíveis)

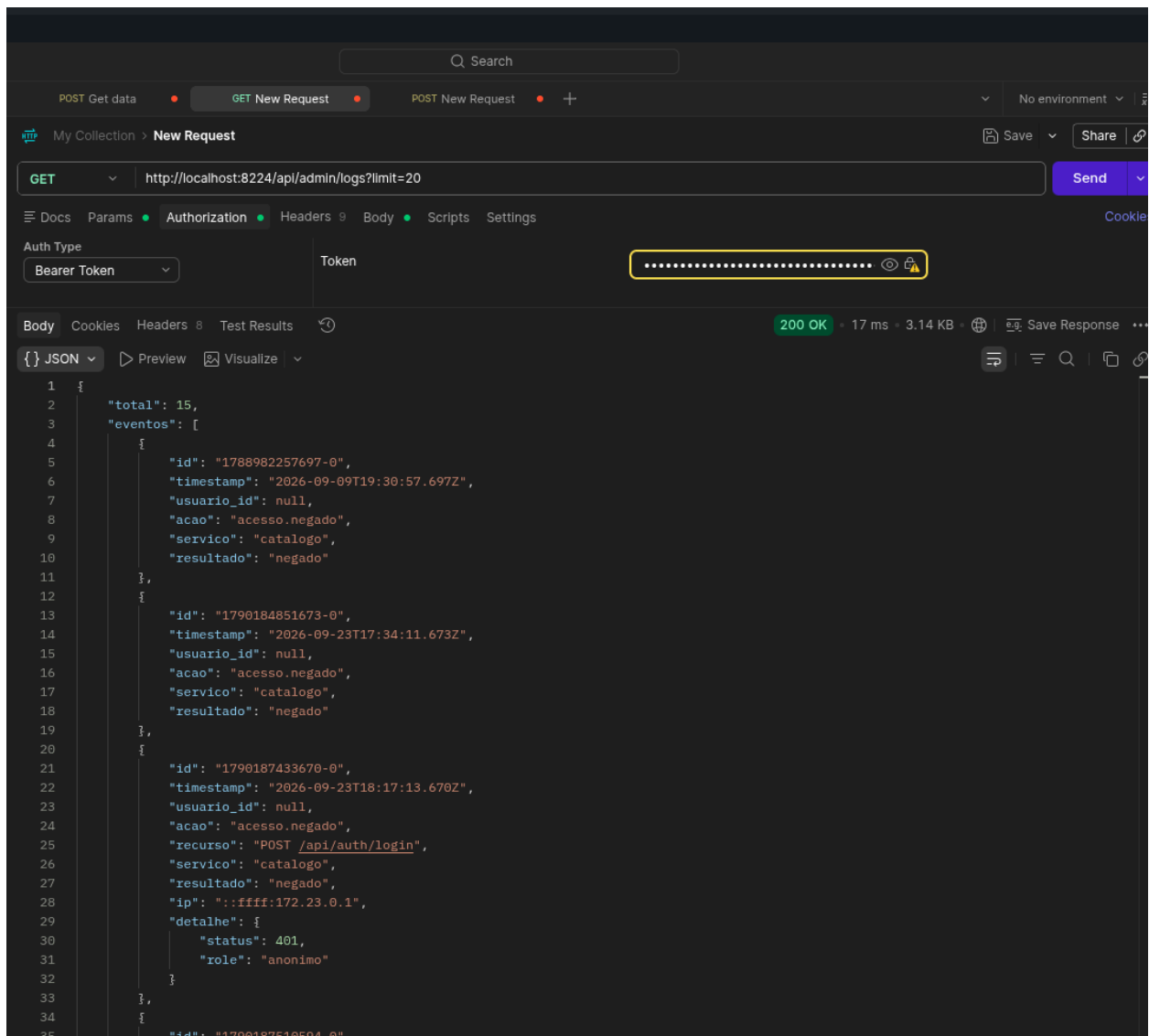


Figura 8: Atividade 5 — resultado (prints dos logs)

Dificuldades e como foram resolvidas. Evitar que uma falha no sistema de logs impactasse a navegação no catálogo. Resolvido tornando a comunicação totalmente assíncrona. O Redis Streams foi escolhido por sua resiliência na ordem cronológica.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	02/10/2026 12:47
Situação	entregue
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. Evolução do catálogo para uma “rede social”. Adicionou-se o armazenamento de fotos como arquivos binários em um Object Storage (MinIO local via Docker), salvando-se apenas a URL no banco MariaDB.

```
~/dev/Projeto-Tom_Hanks on main x > git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" 1a3566b
1a3566b 02/10/2026 12:47
```

Figura 9: Atividade 6 — evidência da entrega pelo terminal (link, data e hora visíveis)

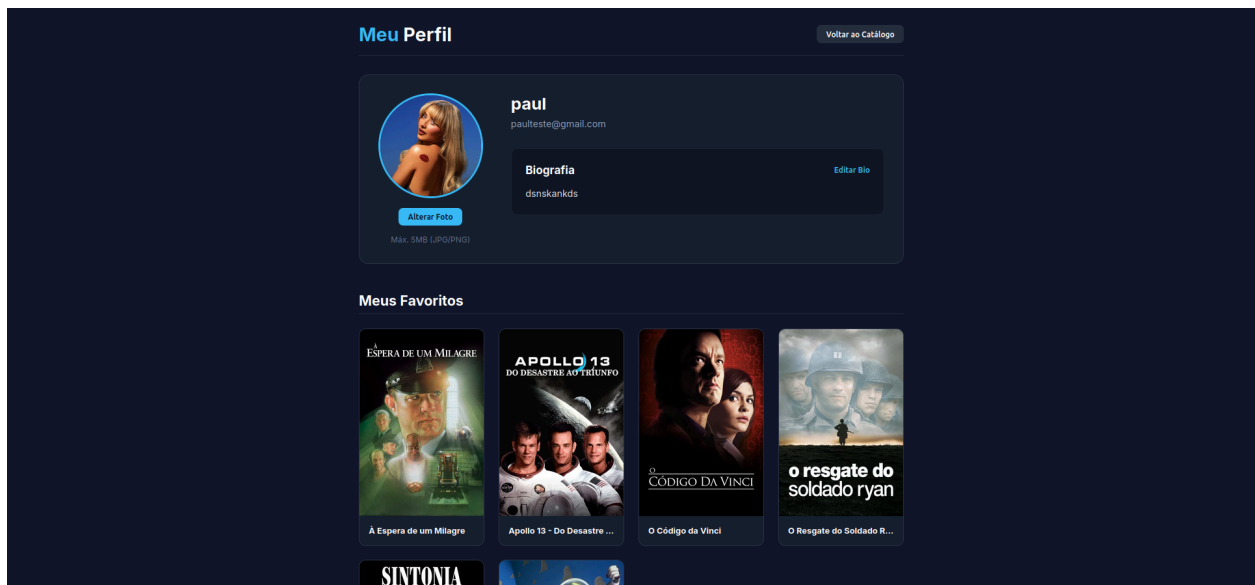


Figura 10: Atividade 6 — resultado (perfil com foto funcionando)

Dificuldades e como foram resolvidas. Conflitos de mapeamento de portas no deploy com o Portainer. A resolução envolveu ajustar a porta externa para 9005 e configurar a criação do bucket com política “Public Read” diretamente via SDK do MinIO no Node.js.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	24/09/2026 18:03
Situação	entregue
Evidência	Terminal — print do comando git log com data e hora exatas
Link	https://github.com/Paulo794/api-filmes-allan

O que foi feito. Implementação de uma interface interativa de documentação da API usando Swagger UI e a especificação OpenAPI, documentando as rotas do catálogo e do auth-service.

```
~/dev/Projeto-Tom_Hanks on main x > git log -1 --date=format:"%d/%m/%Y %H:%M" --format="%h %ad" f58d5bd
f58d5bd 24/09/2026 18:03
```

Figura 11: Atividade E1 — evidência da entrega pelo terminal (link, data e hora visíveis)

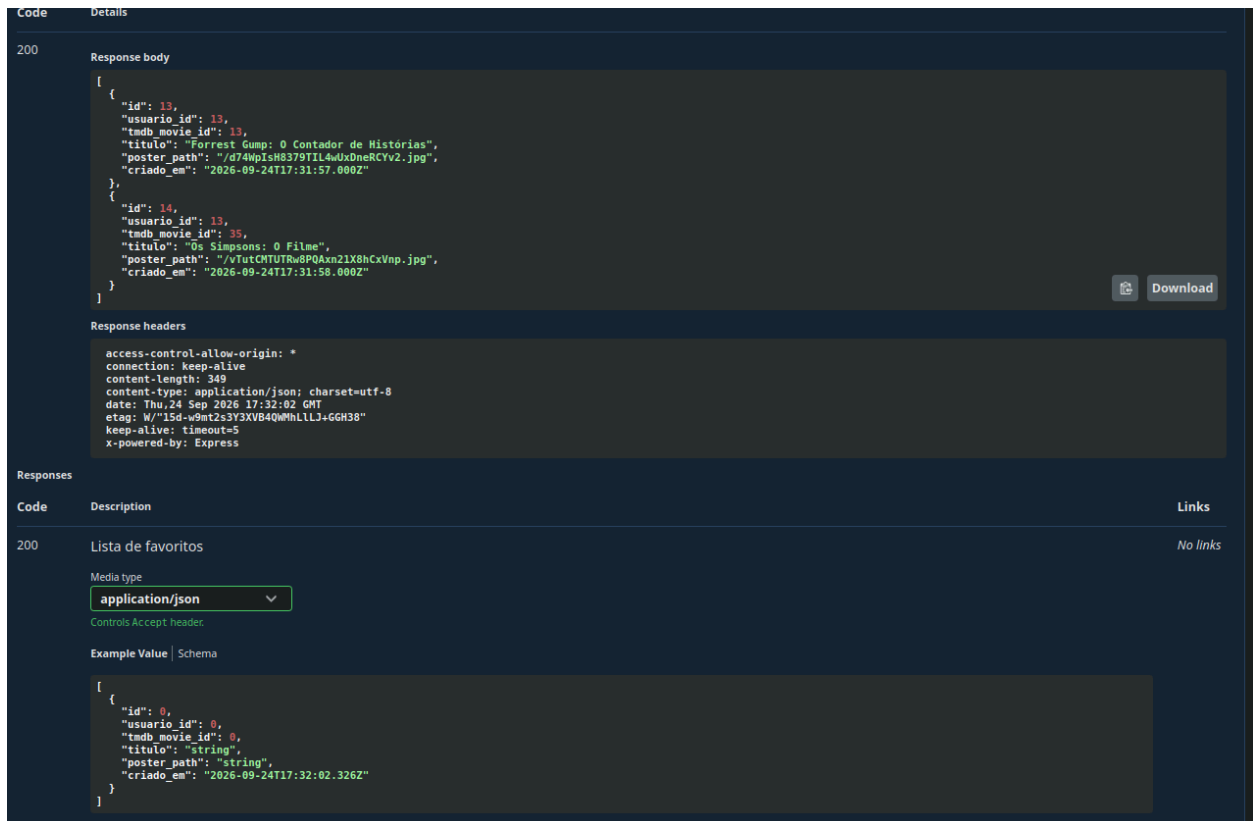


Figura 12: Atividade E1 — resultado (Swagger rodando)

Dificuldades e como foram resolvidas. Configurar o parser de YAML para que o Express conseguisse ler o arquivo OpenAPI corretamente e servir a interface estática do Swagger. Resolvido com os pacotes `swagger-ui-express` e `yamljs`.

6 Considerações finais

Durante este bimestre, o aprendizado mais marcante foi a transição mental do desenvolvimento monolítico direto para a visão complexa de sistemas distribuídos. Trabalhar com microsserviços revelou, na prática, que a comunicação em rede entre containers e o isolamento de ambientes no Docker Compose são tão importantes quanto a própria lógica de programação da aplicação.

A maior dificuldade técnica encontrada ao longo do trajeto foi lidar com o orquestramento de múltiplos serviços de infraestrutura atuando simultaneamente (banco MariaDB, Redis para eventos, armazenamento MinIO e as APIs Node.js). O mapeamento de redes internas e a lida com os conflitos de ambiente remoto demandaram bastante paciência e depuração.

Se eu fosse começar o projeto hoje com o conhecimento já consolidado, eu organizaria a estrutura dos microsserviços de forma independente desde o dia 1, evitando o trabalho de fatiar um sistema que já estava em andamento. O aprendizado sobre infraestrutura como código com Docker mostrou o nível de exigência real em segurança e otimização que o mercado de engenharia de software espera.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Paulo Henrique Domingues Antunes

Pompeia, 07/10/2026